

# AI Agent 面试题总结

## 面试官真正想考什么

Agent 题本质上在考“复杂 AI 应用怎么编排”。可以按下面几个层次准备。

| 考察方向                | 面试官想确认什么                            | 常见扣分点                              |
|---------------------|-------------------------------------|------------------------------------|
| Agent 基础            | 你能否讲清 Agent、Workflow、普通 Chatbot 的区别 | 把 Agent 说成“会自动思考的机器人”              |
| Agent Loop          | 你是否理解推理、行动、观察、修正的循环                 | 只讲工具调用，不讲观察和迭代                     |
| Context Engineering | 你是否知道上下文质量决定 Agent 表现               | 只会调 Prompt，不会管理上下文                 |
| Memory              | 你是否能区分短期状态、长期事实和经验沉淀                | 把历史聊天记录等同于记忆系统                     |
| Tools/MCP/Skills    | 你是否知道工具接入、调用意图和任务 SOP 的边界           | 把 MCP、Function Calling、Skills 混为一谈 |
| Workflow/Harness    | 你是否具备生产级 Agent 工程化思维                | 盲目追求纯 Agent，不考虑可控性                 |

回答 Agent 题时，建议少讲“智能”，多讲“约束”。因为真实项目里，Agent 最大的问题不是不会做事，而是不稳定、不可控、难排查、成本高。

## Agent 基础

参考文章：《AI Agent 核心概念：Agent Loop、Context Engineering、Tools 注册》

这一组题是 Agent 面试的入口。重点不是背公式，而是讲清 Agent 和传统程序、Workflow 的边界。

建议掌握这些关键点：

- Agent 可以理解为 LLM + Planning + Memory + Tools 的组合，但这个公式只是起点，不是完整生产架构。
- 普通 Chatbot 主要回答问题，Agent 更强调多步骤任务执行和外部工具调用。
- Workflow 的路径更固定，适合流程清晰、需要可控性的场景；纯 Agent 更适合路径难提前穷举的开放任务。
- ReAct、Plan-and-Execute、Reflection、Multi-Agent 不是越复杂越好，要结合任务复杂度、调试成本和容错要求选择。

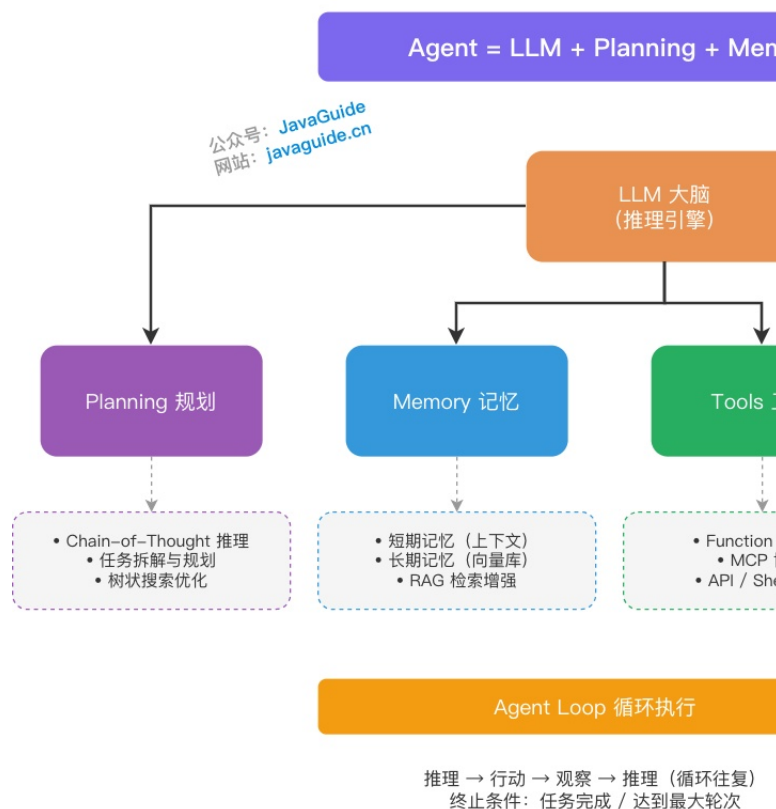
高频面试题：

- AI Agent 是什么？和普通 Chatbot 有什么区别？
- Agent = LLM + Planning + Memory + Tools 这条公式怎么理解？
- Agent Loop 的完整流程是什么？
- Agent 和传统编程、Workflow 的核心区别是什么？
- ReAct、Plan-and-Execute、Reflection、Multi-Agent 分别适合什么场景？
- Tools 注册时，工具 description 为什么很关键？
- 什么时候用纯 Agent，什么时候用 Workflow 或 Agentic Workflow？
- Multi-Agent 协作的主要问题是什么？为什么生产里不能盲目上多 Agent？

一个更稳的回答方式是：先承认 Agent 的动态决策能力，再补上它的代价。比如纯 Agent 灵活，但调试难、轨迹不稳定、Token 成本高；Workflow 可控，但前期流程拆解

要求高。To B 场景通常会优先选择 Workflow 或 Agentic Workflow，把关键路径控制住，只在必要节点让模型做判断。

## AI Agent 核心架构



## AI Agent 核心架构

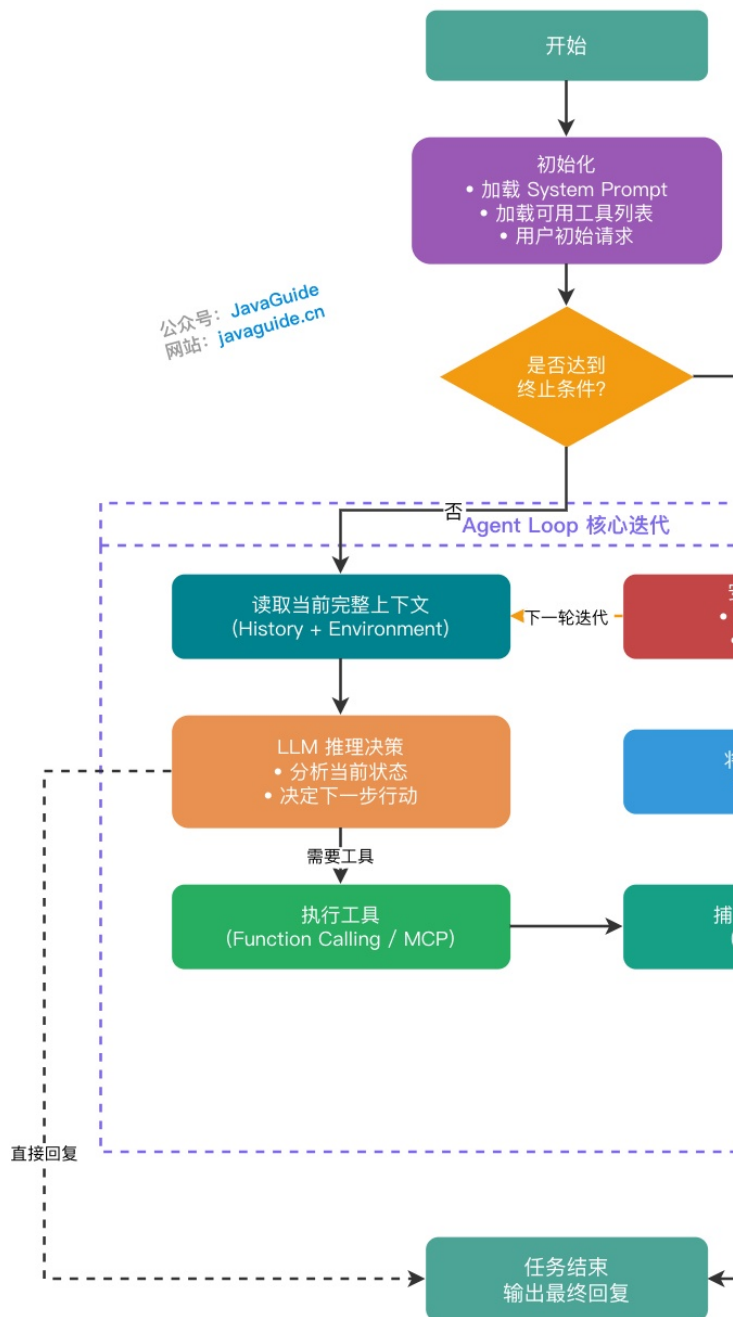
## Agent Loop 工作流程

- 团队共享记忆最好走 Git、PR 和 Review，便于审计和回滚。

高频面试题：

- Agent 的短期记忆和长期记忆有什么区别？
- Agent 记忆系统要解决哪些核心问题？
- 向量记忆和 Markdown 记忆分别适合什么场景？
- Auto Memory 是什么？它为什么不能无限自动写入？
- 团队共享记忆为什么适合走 Git 和 Code Review？
- 记忆压缩、记忆过期、记忆冲突应该怎么处理？
- 如何避免长期记忆污染上下文？
- 面试里怎么讲“有记忆”不是简单保存聊天记录？

如果被追问“怎么设计记忆系统”，可以按读写链路回答：先定义哪些信息允许写入，再做敏感信息过滤和去重；写入时记录来源、时间、置信度和作用域；读取时根据任务检索相关记忆，而不是全量注入；过期或冲突时通过人工审核或规则策略处理。



Agent Loop 工作流程

## Agent Memory

参考文章：《AI Agent 记忆系统：短期记忆、长期记忆与记忆演化机制》

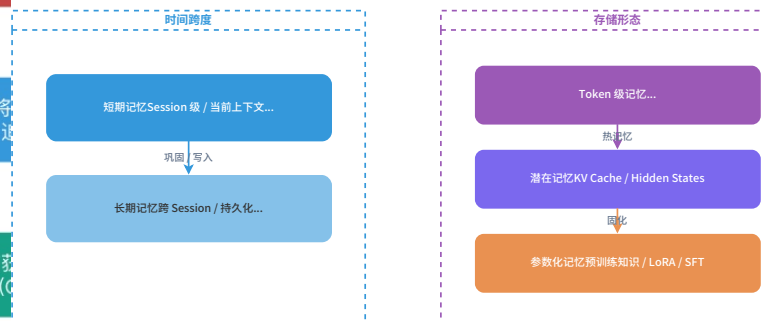
Memory 题经常被问得很细，因为它能区分“玩过 Demo”和“做过系统”的候选人。真正的记忆系统不是把聊天记录一股脑塞回上下文，而是对信息进行分层、筛选、压缩、更新和治理。

建议掌握这些关键点：

- 短期记忆更像当前任务状态，负责记录这一轮任务里必须保留的信息。
- 长期记忆更像跨会话知识，负责沉淀用户偏好、团队规则、历史决策和经验。
- 向量记忆适合语义检索，Markdown 记忆适合规则、偏好、项目约定这类可读可审查的信息。
- 记忆写入不能完全放任模型自动决定，否则容易写入错误、过时、重复或敏感信息。

## Agent 记忆分类全景图

从时间跨度、存储形态、功能目的三个维度理解记忆系统



Agent 记忆分类全景图

## Prompt 与 Context Engineering

参考文章：《大模型提示词工程实践指南》、《上下文工程实践指南：让 Agent 少犯蠢的工程方法论》

Agent 场景下，Prompt 只是入口，Context 才是持续影响模型行为的“工作台”。很多 Agent 不稳定，不是 Prompt 写得不够长，而是上下文里噪声太多、关键约束位置太差、工具结果格式混乱、历史状态没有结构化。

建议掌握这些关键点：

- Prompt Engineering 关注指令怎么写清楚，Context Engineering 关注什么信息在什么时机进入模型窗口。
- Agent 上下文通常包含系统规则、任务目标、历史状态、工具说明、工具结果、用户偏好、检索证据和中间计划。
- 长任务要做上下文压缩、结构化笔记、任务状态持久化和必要的 Sub-agent 拆分。
- Prompt 注入不能只靠提醒模型“不要听用户恶意指令”，还要靠权限隔离、工具白名单、输出校验和审计。

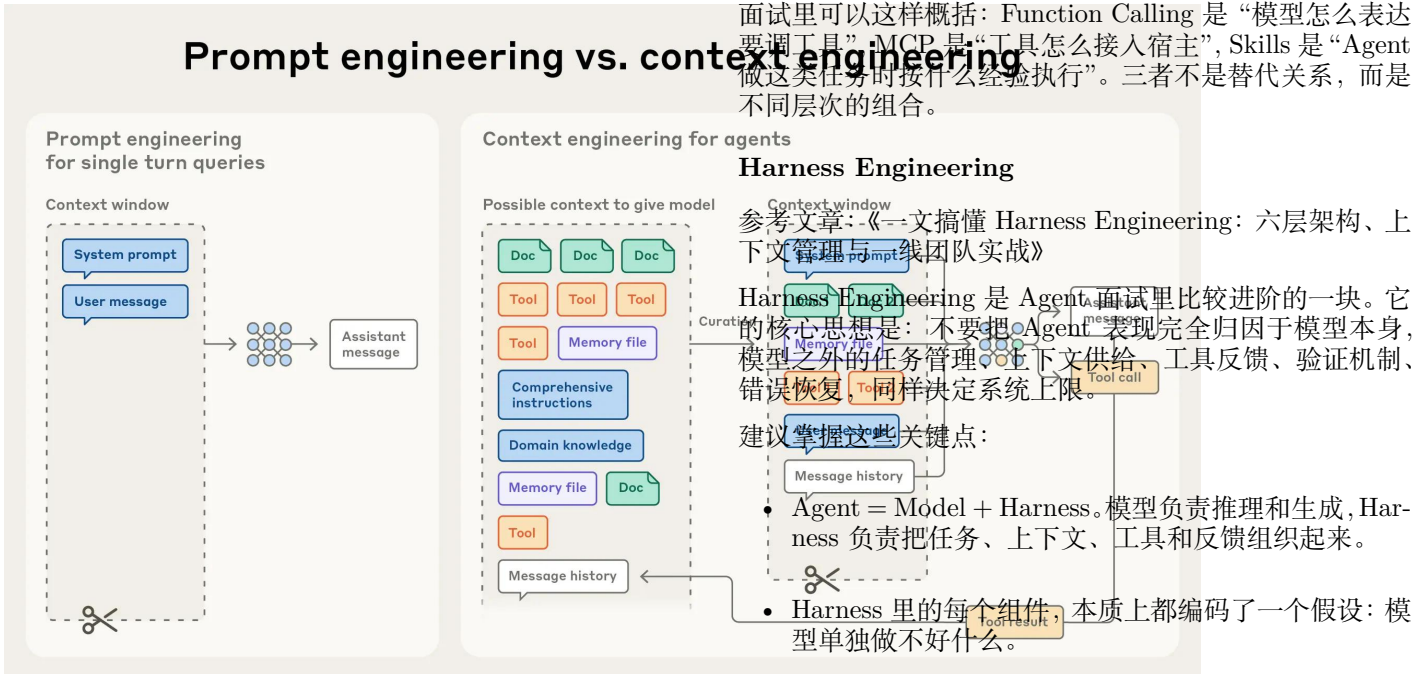
高频面试题：

- Prompt Engineering 和 Context Engineering 有什么区别？
- Prompt 四要素 Role、Task、Context、Format 分别解决什么问题？
- Few-Shot、CoT、任务分解、结构化输出分别适合什么场景？
- Prompt 注入攻击是什么？常见防护方式有哪些？
- 为什么 Agent 场景下只优化 Prompt 不够？

- Context Engineering 要解决哪些问题？
- 静态规则、动态信息、工具结果、记忆应该如何进入上下文？
- 长任务上下文溢出时，Compaction、结构化笔记、Sub-agent 分别怎么用？

答这类题时，可以抓住一句话：**Prompt 决定模型收到什么指令，Context 决定模型实际看到什么世界。**Agent 一旦进入多轮工具调用，后者往往更重要。

- Agent Skills 是什么？它和 Prompt、MCP、Function Calling 的边界是什么？
- Skills 为什么要延迟加载？
- Skill 路由怎么做？为什么它和 RAG 相似但目标不同？
- 写一个 SKILL.md 最容易踩哪些坑？



## Prompt engineering vs. context engineering

### MCP 与 Agent Skills

参考文章：《深入理解 MCP 协议：一次开发，多处复用》、《Agent Skills 是什么？和 Prompt、MCP 到底差在哪？》

这一组题考的是工具生态和能力复用。很多人会把 MCP、Function Calling、Skills 都说成“工具调用”，这样答会显得边界不清。

建议掌握这些关键点：

- Function Calling 解决的是模型如何输出结构化工具调用意图。
- MCP 解决的是工具如何被标准化发现、描述、调用和返回结果。
- Skills 解决的是 Agent 做某类任务时，应该按什么经验和流程执行。
- MCP 更像能力接口，Skills 更像任务 SOP。二者可以组合使用。
- 生产级工具接入必须有权限、参数校验、审计、超时、重试和降级策略。

高频面试题：

- MCP 解决什么问题？为什么常被类比成 AI 领域的 USB-C？
- MCP Client、MCP Server、Host 分别是什么？
- MCP 的 Tools、Resources、Prompts 分别解决什么问题？
- MCP 和 Function Calling 有什么区别？
- 生产级 MCP Server 要做哪些安全治理？

面试里可以这样概括：Function Calling 是“模型怎么表达要调工具”，MCP 是“工具怎么接入宿主”，Skills 是“Agent 做这类任务时按什么经验执行”。三者不是替代关系，而是不同层次的组合。

### Harness Engineering

参考文章：《一文搞懂 Harness Engineering: 六层架构、上下文管理与一线团队实战》

Harness Engineering 是 Agent 面试里比较进阶的一块。它的核心思想是：不要把 Agent 表现完全归因于模型本身，模型之外的任务管理、上下文供给、工具反馈、验证机制、错误恢复，同样决定系统上限。

建议掌握这些关键点：

- Agent = Model + Harness. 模型负责推理和生成, Harness 负责把任务、上下文、工具和反馈组织起来。
- Harness 里的每个组件，本质上都编码了一个假设：模型单独做不好什么。

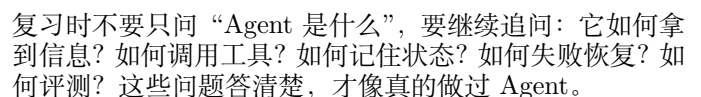
- 模型能力升级后，Harness 也要重新评估。有些过去必要的补丁，可能会变成新的复杂度。

- 上下文污染、代码熵积累、工具调用可靠性，是一线 Agent 工程里很常见的三类问题。

高频面试题：

- Harness Engineering 是什么？它和 Prompt Engineering、Context Engineering 有什么关系？
- 为什么说 Agent = Model + Harness？
- Harness 的六层架构分别解决什么问题？
- 模型能力升级后，Harness 里的某些机制为什么需要重新验证？
- 上下文污染、代码熵积累、工具调用可靠性分别怎么治理？
- Agent 工程里为什么需要评测器、验证器和任务状态管理？
- 一线团队做 Agent 工程化时，共同遇到的难点是什么？

回答时别把 Harness 讲成新名词堆砌。更好的方式是用具体问题带出来：Agent 长任务中途跑偏，需要任务状态和阶段性检查；工具返回错误，模型需要可修复的错误反馈；代码生成重复实现已有逻辑，需要检索和去重机制。这些都是 Harness 要补的系统能力。



面试官如果问“你会怎么设计一个复杂 Agent 流程”，可以先画出固定主链路，再说明哪些节点由模型判断，哪些节点必须由规则和代码控制。这样比直接说“让 Agent 自己规划”可信得多。